Exercice 1 : Gestion d'une bibliothèque universitaire

Analyse, modélisation UML et implémentation C++ (4 classes)

Énoncé. Une bibliothèque universitaire souhaite informatiser la gestion de ses livres, de ses étudiants et des emprunts. Un livre a un identifiant, un titre, un auteur, une année de publication et un état (disponible ou emprunté). Un étudiant a un matricule, un nom, un prénom et une adresse électronique. Un étudiant peut emprunter plusieurs livres, mais un livre ne peut être emprunté que par un seul étudiant à la fois.

1. Classes

Classe	Rôle
Livre	Un ouvrage de la bibliothèque
Etudiant	Un étudiant inscrit
Emprunt	Relie un étudiant à un livre, avec une date d'emprunt et une date de retour
Bibliotheque	Gère les livres, les étudiants et les emprunts ; orchestre les opérations

La classe **Emprunt** est une classe d'association : elle conserve l'historique, alors qu'un simple lien Etudiant–Livre le perdrait au retour du livre.

2. Attributs et méthodes

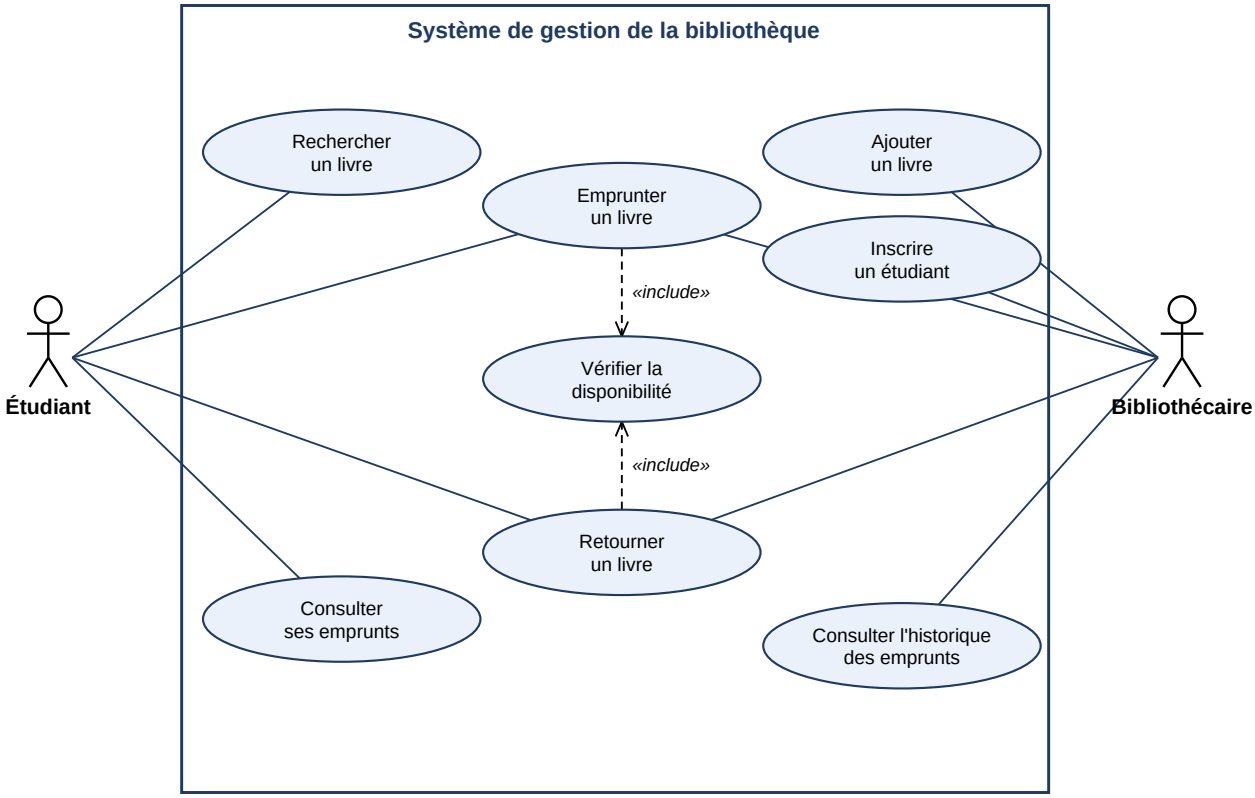
Classe	Attributs (privés)	Méthodes
Livre	id, titre, auteur, annee, disponible	constructeur, getters, estDisponible(), marquerEmprunte(), marquerRendu(), afficher()
Etudiant	matricule, nom, prenom, email, emprunts (liste de pointeurs vers Emprunt)	constructeur, getters, ajouterEmprunt(), nbEmpruntsEnCours(), afficher()
Emprunt	id, livre, etudiant, dateEmprunt, dateRetour, enCours	constructeur, getters, estEnCours(), terminer(date), afficher()
Bibliotheque	nom, livres, etudiants, emprunts, prochainIdEmprunt	ajouterLivre(), ajouterEtudiant(), rechercherLivre(), rechercherEtudiant(), emprunterLivre(), retournerLivre(), afficherLivres(), afficherEtudiants(), afficherEmprunts()

3. Associations et cardinalités

Association	Cardinalités	Type
Etudiant – Emprunt (« effectue »)	1 — 0..*	Association
Livre – Emprunt (« concerne »)	1 — 0..*	Association
Bibliotheque – Livre (« contient »)	1 — 0..*	Agrégation
Bibliotheque – Etudiant (« inscrit »)	1 — 0..*	Agrégation
Bibliotheque – Emprunt (« enregistre »)	1 — 0..*	Composition

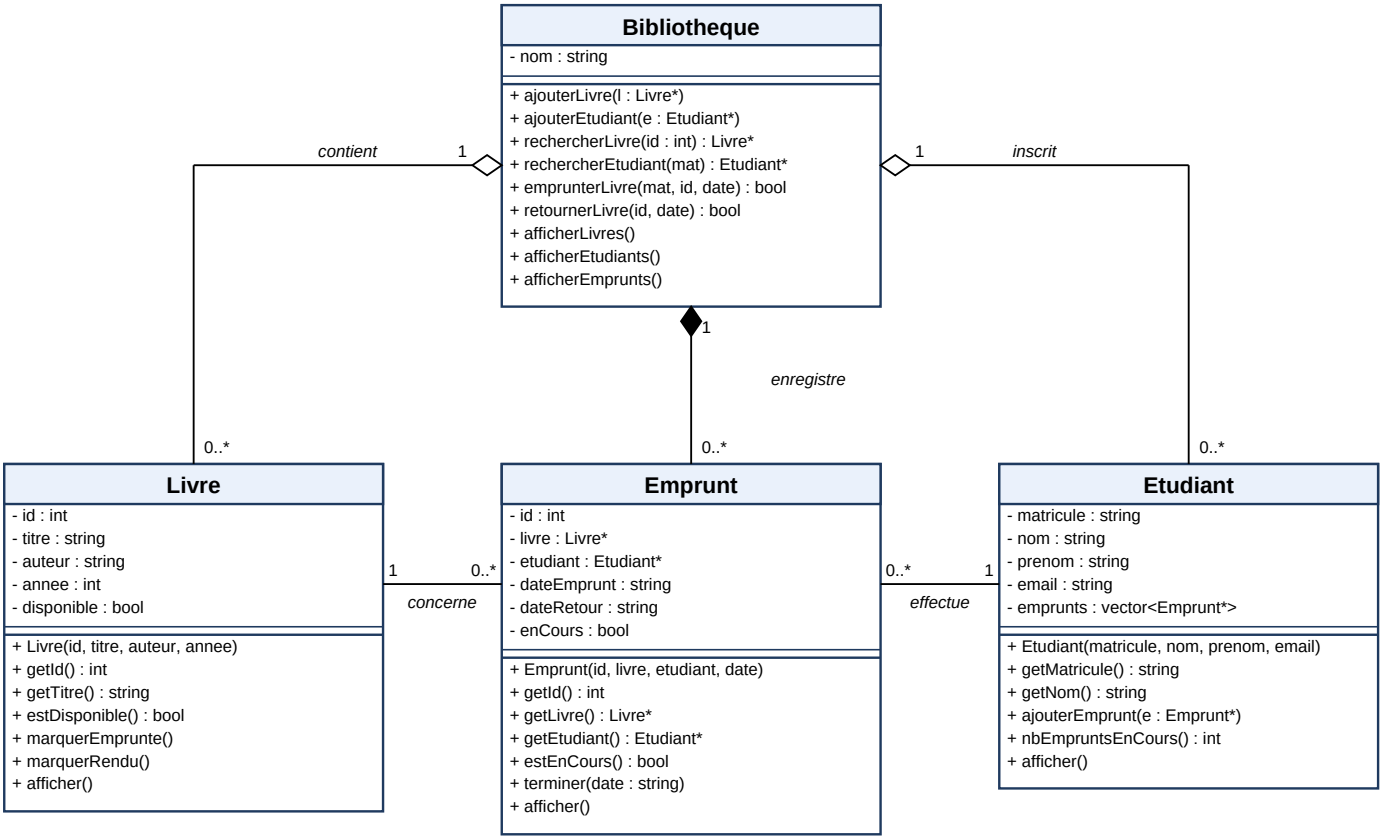
Contrainte métier { un seul emprunt en cours par livre } : un livre ne peut avoir qu'un seul emprunt dont *enCours* est vrai. Un étudiant, lui, peut avoir plusieurs emprunts en cours.

4. Diagramme de cas d'utilisation (UML)



Acteurs : **Étudiant** et **Bibliothécaire**. « Emprunter un livre » et « Retourner un livre » incluent « Vérifier la disponibilité ».

5. Diagramme de classes (UML)



Légende : losange vide = agrégation ; losange plein = composition ; trait simple = association. Le diagramme respecte la contrainte { un seul emprunt en cours par livre } portée par la classe Emprunt.

6. Implémentation C++

Fichier `biblio.cpp` (compilation : `g++ -std=c++11 -o biblio biblio.cpp`).

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Etudiant;  // déclaration anticipée

// ======================== Livre ========================
class Livre {
private:
    int id;
    string titre;
    string auteur;
    int annee;
    bool disponible;

public:
    Livre(int id, const string& titre, const string& auteur, int annee)
        : id(id), titre(titre), auteur(auteur), annee(annee), disponible(true) {}

    int getId() const { return id; }
    string getTitre() const { return titre; }
    string getAuteur() const { return auteur; }
    int getAnnee() const { return annee; }
    bool estDisponible() const { return disponible; }

    void marquerEmprunte() { disponible = false; }
    void marquerRendu() { disponible = true; }

    void afficher() const {
        cout << "[" << id << "] " << titre << " - " << auteur
             << " (" << annee << ") : "
             << (disponible ? "disponible" : "emprunté") << endl;
    }
};

// ======================== Emprunt ========================
class Emprunt {
private:
    int id;
    Livre* livre;
    Etudiant* etudiant;
    string dateEmprunt;
    string dateRetour;
    bool enCours;

public:
    Emprunt(int id, Livre* livre, Etudiant* etudiant, const string& date)
        : id(id), livre(livre), etudiant(etudiant),
          dateEmprunt(date), dateRetour(""), enCours(true) {}

    int getId() const { return id; }
    Livre* getLivre() const { return livre; }
    Etudiant* getEtudiant() const { return etudiant; }
    bool estEnCours() const { return enCours; }

    void terminer(const string& date) {
        dateRetour = date;
        enCours = false;
    }

    void afficher() const;  // défini après Etudiant
};

// ======================== Etudiant ========================
class Etudiant {
private:
    string matricule;
    string nom;
    string prenom;
    string email;
    vector<Emprunt*> emprunts;   // 0..*

public:
    Etudiant(const string& matricule, const string& nom,
             const string& prenom, const string& email)
        : matricule(matricule), nom(nom), prenom(prenom), email(email) {}

    string getMatricule() const { return matricule; }
```

```
    string getNom() const { return nom; }
    string getPrenom() const { return prenom; }
    string getEmail() const { return email; }

    void ajouterEmprunt(Emprunt* e) { emprunts.push_back(e); }

    int nbEmpruntsEnCours() const {
        int n = 0;
        for (const Emprunt* e : emprunts)
            if (e->estEnCours()) n++;
        return n;
    }

    void afficher() const {
        cout << matricule << " - " << prenom << " " << nom << " <" << email
             << "> (" << nbEmpruntsEnCours() << " emprunt(s) en cours)" << endl;
        for (const Emprunt* e : emprunts)
            if (e->estEnCours())
                cout << "     * " << e->getLivre()->getTitre() << endl;
    }
};

void Emprunt::afficher() const {
    cout << "Emprunt #" << id << " : \"" << livre->getTitre() << "\" par "
         << etudiant->getPrenom() << " " << etudiant->getNom()
         << " | du " << dateEmprunt << " au "
         << (enCours ? string("(en cours)") : dateRetour) << endl;
}

// ======================== Bibliotheque ========================
class Bibliotheque {
private:
    string nom;
    vector<Livre*> livres;
    vector<Etudiant*> etudiants;
    vector<Emprunt*> emprunts;   // composition : la bibliothèque les possède
    int prochainIdEmprunt;

public:
    Bibliotheque(const string& nom) : nom(nom), prochainIdEmprunt(1) {}

    ~Bibliotheque() {
        for (Emprunt* e : emprunts) delete e;
    }

    void ajouterLivre(Livre* l) { livres.push_back(l); }
    void ajouterEtudiant(Etudiant* e) { etudiants.push_back(e); }

    Livre* rechercherLivre(int id) const {
        for (Livre* l : livres) if (l->getId() == id) return l;
        return nullptr;
    }

    Etudiant* rechercherEtudiant(const string& mat) const {
        for (Etudiant* e : etudiants) if (e->getMatricule() == mat) return e;
        return nullptr;
    }

    bool emprunterLivre(const string& mat, int idLivre, const string& date) {
        Etudiant* e = rechercherEtudiant(mat);
        Livre* l = rechercherLivre(idLivre);
        if (!e || !l) {
            cout << "Étudiant ou livre introuvable.\n";
            return false;
        }
        if (!l->estDisponible()) {
            cout << "Échec : \"" << l->getTitre() << "\" est déjà emprunté.\n";
            return false;
        }
        Emprunt* emp = new Emprunt(prochainIdEmprunt++, l, e, date);
        emprunts.push_back(emp);
        e->ajouterEmprunt(emp);
        l->marquerEmprunte();
        cout << "OK : " << e->getNom() << " a emprunté \"" << l->getTitre() << "\".\n";
        return true;
    }

    bool retournerLivre(int idLivre, const string& date) {
        Livre* l = rechercherLivre(idLivre);
        if (!l) {
            cout << "Livre introuvable.\n";
            return false;
        }
```

```
        for (Emprunt* emp : emprunts) {
            if (emp->getLivre() == l && emp->estEnCours()) {
                emp->terminer(date);
                l->marquerRendu();
                cout << "OK : \"" << l->getTitre() << "\" a été retourné.\n";
                return true;
            }
        }
        cout << "Ce livre n'est pas emprunté.\n";
        return false;
    }

    void afficherLivres() const {
        cout << "\n--- Livres de " << nom << " ---\n";
        for (const Livre* l : livres) l->afficher();
    }
    void afficherEtudiants() const {
        cout << "\n--- Étudiants ---\n";
        for (const Etudiant* e : etudiants) e->afficher();
    }
    void afficherEmprunts() const {
        cout << "\n--- Historique des emprunts ---\n";
        for (const Emprunt* e : emprunts) e->afficher();
    }
};

// ======================== main ========================
int main() {
    Livre l1(1, "Le Petit Prince", "A. de Saint-Exupéry", 1943);
    Livre l2(2, "Les Misérables", "Victor Hugo", 1862);
    Livre l3(3, "Une si longue lettre", "Mariama Bâ", 1979);

    Etudiant e1("M001", "Adjovi", "Cyriande", "cyriande@univ.bj");
    Etudiant e2("M002", "Koffi", "Jean", "jean.koffi@univ.bj");

    Bibliotheque bib("Bibliothèque universitaire");
    bib.ajouterLivre(&l1); bib.ajouterLivre(&l2); bib.ajouterLivre(&l3);
    bib.ajouterEtudiant(&e1); bib.ajouterEtudiant(&e2);

    bib.afficherLivres();

    cout << "\n=== Emprunts ===\n";
    bib.emprunterLivre("M001", 1, "12/11/2026");
    bib.emprunterLivre("M001", 2, "12/11/2026");  // plusieurs livres : autorisé
    bib.emprunterLivre("M002", 1, "12/11/2026");  // refusé : déjà emprunté

    bib.afficherLivres();
    bib.afficherEtudiants();

    cout << "\n=== Retour ===\n";
    bib.retournerLivre(1, "19/11/2026");
    bib.emprunterLivre("M002", 1, "20/11/2026");  // maintenant possible

    bib.afficherLivres();
    bib.afficherEtudiants();
    bib.afficherEmprunts();
    return 0;
}
```

7. Exécution du programme

Scénario : M001 emprunte les livres 1 et 2 ; M002 tente d'emprunter le livre 1 (refusé) ; le livre 1 est retourné ; M002 peut alors l'emprunter.

```
--- Livres de Bibliothèque universitaire ---
[1] Le Petit Prince - A. de Saint-Exupéry (1943) : disponible
[2] Les Misérables - Victor Hugo (1862) : disponible
[3] Une si longue lettre - Mariama Bâ (1979) : disponible

=== Emprunts ===
OK : Adjovi a emprunté "Le Petit Prince".
OK : Adjovi a emprunté "Les Misérables".
Échec : "Le Petit Prince" est déjà emprunté.

--- Livres de Bibliothèque universitaire ---
[1] Le Petit Prince - A. de Saint-Exupéry (1943) : emprunté
[2] Les Misérables - Victor Hugo (1862) : emprunté
[3] Une si longue lettre - Mariama Bâ (1979) : disponible

--- Étudiants ---
M001 - Cyriande Adjovi <cyriande@univ.bj> (2 emprunt(s) en cours)
     * Le Petit Prince
     * Les Misérables
M002 - Jean Koffi <jean.koffi@univ.bj> (0 emprunt(s) en cours)

=== Retour ===
OK : "Le Petit Prince" a été retourné.
OK : Koffi a emprunté "Le Petit Prince".

--- Livres de Bibliothèque universitaire ---
[1] Le Petit Prince - A. de Saint-Exupéry (1943) : emprunté
[2] Les Misérables - Victor Hugo (1862) : emprunté
[3] Une si longue lettre - Mariama Bâ (1979) : disponible

--- Étudiants ---
M001 - Cyriande Adjovi <cyriande@univ.bj> (1 emprunt(s) en cours)
     * Les Misérables
M002 - Jean Koffi <jean.koffi@univ.bj> (1 emprunt(s) en cours)
     * Le Petit Prince

--- Historique des emprunts ---
Emprunt #1 : "Le Petit Prince" par Cyriande Adjovi | du 12/11/2026 au 19/11/2026
Emprunt #2 : "Les Misérables" par Cyriande Adjovi | du 12/11/2026 au (en cours)
Emprunt #3 : "Le Petit Prince" par Jean Koffi | du 20/11/2026 au (en cours)
```

8. Notions mobilisées

- **Classes et objets** : 4 classes, instanciées dans *main*.
- **Encapsulation** : tous les attributs sont *private* ; l'accès passe par des getters et des méthodes métier.
- **Constructeurs** : listes d'initialisation ; un Emprunt démarre toujours avec *enCours* = *true*.
- **Associations** : *vector<Emprunt*>* côté Etudiant, *Livre** et *Etudiant** côté Emprunt ; composition Bibliotheque–Emprunt (création avec *new*, libération dans le destructeur).
- **Règle métier** : *emprunterLivre()* refuse l'emprunt si *estDisponible()* est faux, donc un livre n'a jamais deux emprunts en cours.